

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК.01.04 Системное программирование

ПЦК Программирование

ВИД УЧЕБНОГО
МАТЕРИАЛА Лекция - 1

ПРЕПОДАВАТЕЛЬ Мухортова Наталья Николаевна

Введение и основополагающие концепции

Часть I. Введение в системное программирование

1.1 Что такое системное программирование

1.1.1 Определение и основные отличия от прикладного программирования

Системное программирование — это процесс разработки программного обеспечения, которое взаимодействует с аппаратным обеспечением и операционной системой на низком уровне. Это включает написание кода, который работает непосредственно с ресурсами компьютера, такими как процессор, память, устройства ввода-вывода и сети. В отличие от прикладного программного обеспечения, которое ориентировано на конечного пользователя, системное программное обеспечение (СПО) предназначено для обеспечения функционирования компьютерной системы и других программ.

Пример системного программного обеспечения:

- Операционные системы (ОС) — обеспечивают управление ресурсами компьютера и их эффективное использование.
- Драйверы устройств — позволяют операционной системе взаимодействовать с аппаратными устройствами.
- Компиляторы и интерпретаторы — переводят программы на языке высокого уровня в машинный код, который может быть исполнен процессором.

Основные задачи системного программирования:

- Создание программ, которые взаимодействуют с операционной системой на уровне ядра (например, через системные вызовы).
- Оптимизация использования ресурсов системы, таких как процессор, память, устройства ввода-вывода.
- Обеспечение безопасности и надежности системы.
- Поддержка многозадачности и параллельного выполнения процессов.

1.1.2 Различия между системным и прикладным программированием

1. Уровень взаимодействия с аппаратным обеспечением:
   - В системном программировании программы работают на уровне ядра операционной системы и аппаратного обеспечения. Программисты используют низкоуровневые API и системные вызовы для управления ресурсами системы.
   - В прикладном программировании разработчики создают программное обеспечение, которое работает поверх операционной системы и использует высокоуровневые API.
2. Оптимизация производительности:
   - Системное программирование требует высокой степени оптимизации для достижения максимальной производительности, так как ресурсы системы могут быть ограничены.
   - Прикладное программное обеспечение, как правило, меньше заботится об оптимизации на уровне процессора и памяти, фокусируясь на пользовательском опыте.
3. Управление памятью:
   - В системном программировании программисты часто вручную управляют памятью (например, используя `malloc()` и `free()` в языке C).

- В прикладном программировании управление памятью обычно автоматизировано (например, сборка мусора в языках Java или Python).

1.1.3 Пример: простая программа с использованием системного вызова

Для иллюстрации простейшего примера системного программирования рассмотрим программу на языке C, которая использует системный вызов `write()` для вывода текста в стандартный поток вывода (консоль). Это типичный пример работы с системным вызовом, где происходит прямое обращение к ядру операционной системы для выполнения задачи.

```
ude <unistd.h>


ain() {
nst char *message = "Hello, World!\n";
ite(1, message, 14);  // 1 - это файловый дескриптор для стандартного выво
urn 0;
```

Copy

Объяснение:

- Здесь используется системный вызов `write()`, который напрямую вызывает функции ядра ОС для вывода строки в стандартный поток вывода (консоль). Этот код иллюстрирует основную концепцию системного программирования — прямое взаимодействие с ресурсами системы.

1.2 Роль системного программного обеспечения в ИТ-инфраструктуре

1.2.1 Операционная система как основа системного программного обеспечения

Операционная система (ОС) — это основной компонент системного программного обеспечения, который управляет всеми ресурсами компьютера и предоставляет интерфейс для взаимодействия с прикладным ПО. ОС выполняет следующие функции:

- Управление процессами: ОС организует выполнение нескольких процессов, предоставляя им возможность работать одновременно и эффективно использовать процессор.
- Управление памятью: ОС обеспечивает распределение и защиту памяти между процессами, предотвращая конфликты при доступе к данным.
- Управление устройствами ввода-вывода: ОС взаимодействует с периферийными устройствами (жесткие диски, принтеры, сетевые устройства), абстрагируя прикладные программы от деталей работы с аппаратурой.

- Обеспечение безопасности: ОС гарантирует, что приложения и пользователи не могут получить несанкционированный доступ к данным и ресурсам системы.

1.2.2 Важность системного ПО в современных системах

Системное программное обеспечение играет ключевую роль в современных вычислительных системах, обеспечивая:

- Стабильность и надежность: Без качественного системного ПО все приложения на компьютере могут работать нестабильно или некорректно, что приведет к сбоям и потере данных.
- Производительность: Оптимизация системного ПО позволяет максимально эффективно использовать ресурсы системы (CPU, RAM, диск и т.д.), что особенно важно в условиях высоких нагрузок.
- Безопасность: Системное ПО отвечает за обеспечение доступа к ресурсам системы, контроль привилегий и защиту от атак (например, обеспечение безопасности через мандатное управление доступом, такие как SELinux).

Пример:

- Виртуализация: Системное программное обеспечение играет важную роль в современных облачных системах, предоставляя возможность виртуализации и контейнеризации. Это позволяет разделять ресурсы между различными пользователями и приложениями, обеспечивая изоляцию и безопасность.

1.3 Основные задачи системного программирования

1.3.1 Взаимодействие с аппаратным обеспечением

Одной из ключевых задач системного программирования является создание программ, которые напрямую работают с аппаратными компонентами системы:

- Управление устройствами (например, работа с устройствами хранения, сетевыми картами, принтерами).
- Использование драйверов для взаимодействия с периферией.

Пример:

- Написание драйвера для устройства ввода-вывода, который принимает запросы от ОС и управляет работой конкретного устройства.

1.3.2 Оптимизация использования ресурсов системы

Системное программное обеспечение должно быть эффективно, чтобы минимизировать накладные расходы на использование процессора, памяти и ввода-вывода. Оптимизация включает в себя:

- Управление памятью и устранение утечек памяти.
- Минимизация времени ожидания ввода-вывода (например, асинхронные операции).

Пример:

- Оптимизация программы, использующей многопоточность для работы с сетевыми запросами, чтобы повысить производительность в условиях высокой нагрузки.

1.3.3 Работа с системными вызовами

Системные вызовы — это интерфейс, через который программы могут взаимодействовать с ядром операционной системы. Каждый системный вызов выполняется в привилегированном режиме и позволяет программе запрашивать ресурсы или выполнять определенные действия.

Пример:

- Системные вызовы `open()`, `read()`, `write()`, и `close()` для работы с файлами.

Практическое задание:

1. Написать программу на C, которая создает новый процесс с использованием `fork()`, а затем выполняет другую программу в дочернем процессе с помощью `exec()`. Программа должна ждать завершения дочернего процесса и выводить его результат.

Заключение к главе 1

Мы рассмотрели основные понятия системного программирования, его задачи и роль системного ПО в современных ИТ-системах. Системное программирование является неотъемлемой частью разработки программ,

которые работают непосредственно с ресурсами системы, и требует глубокого понимания работы операционной системы и аппаратного обеспечения. В следующих разделах мы углубимся в конкретные аспекты системного программирования, такие как многопоточность, работа с памятью, системные вызовы и виртуализация.